

Case Mibo Smart

Documento de produto e arquitetura

Documento de entrega, 24/09/2026. O detalhe vive no repositório: [docs/adr/](#) (27 decisões, com as opções recusadas), [docs/specs/SPEC.md](#) (50 critérios de aceite), [docs/api-contract.md](#) (o contrato como observado), [AI-LOG.md](#) (uso de IA).

1. Objetivo

App Kotlin Multiplatform que consome a plataforma Open Casa Inteligente: token de acesso, lista de dispositivos com paginação e filtro, vídeo ao vivo de câmeras e controle de fechadura.

Android é o alvo principal; iOS existe para provar que o compartilhamento é real, não rótulo.

2. Organização da semana

Dia	Foco	Saída
Sáb 19	Stack, bibliotecas, padrão de arquitetura	nenhuma linha de código, de propósito
Dom 20	Sondar a API; ler avaliações das lojas; ver vídeos do app oficial em uso	SPEC, ADR-001 a 008, contrato observado. Primeiro commit às 17h11
Seg 21	Implementação	96 commits
Ter 22	Implementação, identidade visual, testes de tela	26 commits
Qua 23	Implementação, correções contra a API real	21 commits

Dois dias antes da primeira linha. A SPEC e os ADRs existiam antes da primeira fatia, então nenhuma decisão estrutural precisou ser tomada com o prazo em cima.

Ondas e ordem de corte

Onda	Escopo
0	<i>Walking skeleton</i> : CI, esqueleto de módulos, SPEC, ADRs, testes de arquitetura
1	Sessão e token
2	Lista, vídeo ao vivo e fechadura, em paralelo
3	Histórico, renovação de token, iOS, módulo Java, R8

A ordem de corte foi decidida antes de precisar dela: módulo Java → histórico → fallback do vídeo → R8. Vídeo e fechadura nunca foram cortáveis.

A fatia é vertical: atravessa `domain → data → app → tela` e entrega algo observável. Teto de ~400 linhas executáveis de produção por PR, medido por `tools/executable-lines.py`. Testes são contados e não entram no teto, porque quem os limita são os critérios de aceite.

3. O problema, lido antes de escrito

Antes de abrir a primeira issue, li o que quem usa o app oficial escreve: 250 avaliações da App Store, 20 da Play, Reclame Aqui e o fórum oficial (`docs/research/user-feedback.md`, leitura em 20/09). Nota agregada 4,8★ nas duas lojas, mas as avaliações recentes *com texto* são majoritariamente 1★.

O que mais dói	O que o case faz a respeito
Vídeo trava em “9x %” sem erro nomeado (≈50 menções)	Etapas visíveis, timeout de 20 s com mensagem, retry e fallback web. Nunca um spinner infinito
Propaganda e push de marketing num app de segurança (≈45)	Zero banner, pesquisa ou notificação. Abre direto na lista, câmera em dois toques
Câmera “offline do nada”, sem explicação (≈20)	Indicador e “visto pela última vez há X”, na lista e na fechadura
Fechadura: histórico atrasado e sem nome (≈10)	Comando → confirmação explícita; histórico com quem e quando
Erros genéricos (“Erro desconhecido”)	Toda mensagem diz a causa e oferece uma ação; nenhum código HTTP na tela

Os critérios U1–U8 da SPEC nascem daí, cada um com o teste que o prova.

Fora do escopo, registrado como próximo passo de produto: gravações e timeline, notificações de dispositivo com granularidade, uma conta para todas as linhas. E um relato de senha excluída que continuava abrindo a fechadura, que pede tratamento de incidente e não backlog.

4. Arquitetura

```
:shared:domain    Kotlin puro. Modelos, contratos, erros tipados, resultados por caso de uso
:shared:data      O parceiro mora aqui: Ktor, DTOs, mapeadores, cache, cofre do token
:shared:app       Casos de uso, ViewModels, telas Compose. Gera o framework do iOS
:androidApp       Activity e Application. Nenhuma lógica
iosApp            SwiftUI hospedando o framework
:legacy-catalog   Módulo Java, para provar a interoperabilidade
```

Quatro decisões sustentam o resto:

- **Dependências apontam só para dentro.** O `domain` não conhece Ktor, Koin, Compose nem SQLDelight. “Suporte a múltiplos parceiros” vira um segundo módulo de dados implementando os mesmos contratos, sem tocar em tela nem caso de uso (ADR-001, ADR-004).
- **Erro é valor.** A camada de dados lança exceções tipadas; cada caso de uso devolve o resultado da sua intenção. O `when` da tela é exaustivo e o compilador cobra (ADR-002).
- **Um estado imutável por tela.** Estados de hardware são tipos selados, não booleanos combinados (ADR-003).

- `expect/actual` só em pacote `platform`, dentro do módulo dono da abstração (ADR-005, ADR-008).

Como poderia ser aprimorada

- Modularizar **por feature** em vez de por camada, quando o time crescer: hoje o `:shared:app` concentra quatro features e é o arquivo mais disputado num merge.
- Um segundo parceiro de verdade validaria o ADR-004. Hoje a fronteira é boa por construção, não por prova.
- Os goldens de tela como porta de regressão, não como figura anexada ao PR.
- Uma camada de sincronismo se o app ganhar escrita offline. Hoje o cache é só leitura.

5. O que não é escolha padrão

Ktor, Koin e SQLDelight são o previsível de um projeto KMP. O que vale explicar é o resto:

Escolha	Por que entrou
Konture	Fronteira de módulo como teste JVM que falha o build , não como convenção num README
Roborazzi	42 capturas geradas das previews que já existem, com guarda contra deriva
Kover	A meta de 80% de cobertura só existe se for medida
Módulo Java puro	Interoperabilidade provada por um teste escrito em Java , não afirmada
Media3 / WKWebView	<code>expect/actual</code> só na superfície do player; VLCKit recusado por tamanho

Konture e Kover vieram das duas preocupações que o time citou na entrevista: manter a arquitetura ao longo do tempo, e cobertura de testes alta.

Regra que governa a lista: nenhuma dependência entra para “fazer compilar”. Ela entra com uma linha de ADR dizendo o que resolve e o que foi recusado no lugar.

6. A fronteira é um teste

Cinco regras rodam como teste JVM em todo PR, antes de qualquer teste de negócio:

Regra	O que proíbe
1	O <code>domain</code> depender de framework, persistência ou dos módulos de fora
2	Dependência apontando para fora, e ciclo no grafo de módulos
4	<code>android.*</code> , <code>java.*</code> ou <code>javax.*</code> em <code>commonMain</code>
5	Repositório declarado no <code>domain</code> que não seja interface
8	<code>expect / actual</code> fora de um pacote <code>platform</code>

E um **teste negativo permanente**: uma regra que o grafo real viola de propósito tem que falhar. Se ela passar verde, quem quebrou foi a ferramenta, e as outras cinco estão passando por vacuidade.

7. O orçamento de requisições é o eixo do desenho

A conta de teste tem franquia finita e cada chamada conta, inclusive validar o token. Isso deixou de ser restrição e virou princípio:

- Cache da lista: a tela abre com o que já sabe, antes de a rede responder.
- Zero polling, em lugar nenhum do app.
- Formato do token validado localmente, antes de gastar chamada com colagem truncada.
- Trocar o chip de filtro não custa requisição quando a lista completa já está em memória.
- Fechadura: uma leitura de confirmação, com ação do usuário, em vez de polling.

Na véspera da entrega a franquia chegou a zero e a API passou a recusar tudo. O desenho já estava pronto para isso.

8. Estados que o app se recusa a simplificar

Fechadura. Comando e confirmação são coisas separadas, e nenhuma tela afirma que a porta abriu antes de uma leitura concordar. “Enviado e não confirmado” não é “recusou” nem “abriu”, e ninguém consegue distinguir as duas coisas a partir da API (ADR-021). Escrita é opt-in no build: `mudar-volume` e `habilitar-abrir-remoto` terminam em hardware de uma conta compartilhada.

Câmera. Criando sessão, ao vivo, reconectando, cota esgotada, offline, falha com fallback web.

Sessão. Sem token, válida com prazo, expirada com o motivo e retorno à tela anterior.

9. Segurança

Token só em Keystore/Keychain, nunca em preferências simples, logs ou UI (só o sufixo aparece). `Authorization` sanitizado no logger HTTP, e o CI falha se um padrão de token aparecer no repositório. Nenhum host ou credencial versionado. Detalhe no ADR-008.

10. Qualidade

Módulo	Testes
<code>:shared:domain</code>	19
<code>:shared:data</code>	84
<code>:shared:app</code>	195
<code>:konture-test</code>	11
<code>:legacy-catalog</code>	5
Total	314

Cobertura agregada: **61,5 % de linhas, 61 % de ramos**, excluindo por decisão explícita as pontes de plataforma e a UI Compose, que são provadas por preview e golden.

O que barra um merge é o teste de arquitetura, a suíte e a guarda de deriva das telas. Cobertura é medida e publicada, não é porta (ADR-006). O caminho até os 80 % passa pelas 42 capturas de tela virarem porta de regressão, o que depende de uma baseline gravada no CI.

Método: SPEC com critérios EARS → ADR quando a decisão é estrutural → uma issue por fatia vertical → um PR por issue, com o porquê no corpo. 55 PRs mesclados, squash por humano. TDD cirúrgico nas regras críticas; UI por preview e golden.

11. Como rodar

`docs/guides/running.md` tem o passo a passo, incluindo iOS com e sem Mac. O essencial:

```
./gradlew :androidApp:assembleDebug
./gradlew :konture-test:test :shared:domain:testAndroidHostTest \
    :shared:data:testAndroidHostTest :shared:app:testAndroidHostTest
```

Antes: copie `local.properties.example` para `local.properties` e preencha os dois hosts. Sem eles o build falha com mensagem clara, em vez de embutir um padrão. Nenhum teste fala com a API real.

12. O que ficou de fora

O vídeo não chegou a exibir imagem. A sessão é criada normalmente e o fluxo do portal responde `200 video/mp4 chunked`, mas encerra em 15 s sem enviar dados, nas duas câmeras e em todos os canais. O comportamento é do transcodificador do parceiro e foi reportado com as medições. Que o caminho já funcionou é medido: sessões do mesmo dia registraram consumo constante de 0,87 Mbit/s.

Os números de retry do vídeo (1/3/7 s, e 20 s para o primeiro quadro) estão implementados e provados em relógio virtual. O mecanismo é testado; os números não foram observados contra hardware, e seguem marcados como suposição.

A baseline de goldens não está commitada: ela só pode nascer no runner Linux, porque a rasterização de fonte difere entre sistemas operacionais.

Ordenação escolhível da lista não entrou. A ordem fixa de hoje é deliberada e testada: câmeras e fechaduras online primeiro, depois os demais, cada grupo por nome.